

Entscheidungsbäume

Mit Ja/Nein-Fragen vom Datensatz zur Vorhersage

Die Idee

Ein **Entscheidungsbaum** (*Decision Tree*) trifft Vorhersagen, indem er nacheinander einfache Ja/Nein-Fragen stellt – genau wie ein Flussdiagramm. Jede Frage teilt die Daten in zwei Gruppen, bis am Ende eine Vorhersage steht. Der Algorithmus lernt *selbst*, welche Fragen er stellen muss und in welcher Reihenfolge.

Entscheidungsbaum: Überleben auf der Titanic vorhersagen

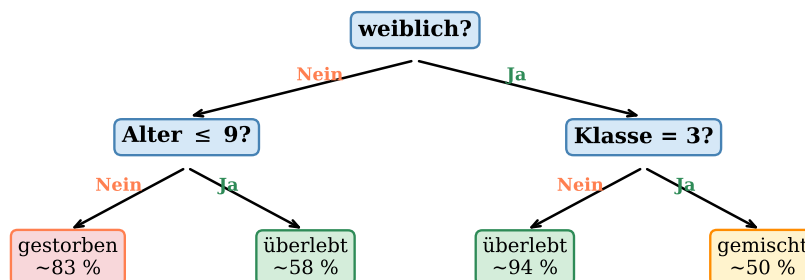


Abbildung 1: Blaue Knoten stellen Fragen, farbige Blätter geben die Vorhersage – der Baum entdeckt selbst die Regel „Frauen und Kinder zuerst“.

Wie wählt der Baum seine Fragen?

An jedem Knoten probiert der Algorithmus *alle* möglichen Fragen aus (z. B. „Alter ≤ 9?“, „Fahrpreis > 50?“, ...) und wählt die mit dem größten **Informationsgewinn**: Nach dem Split sollen die Gruppen möglichst *rein* sein – also einheitlich *überlebt* oder *gestorben*. Reinheit misst man bei Klassifikation mit der **Gini-Unreinheit** $G = 1 - \sum_k p_k^2$, bei Regression mit der Reduktion des RSS. Jede Frage zieht eine gerade Grenze durch den Merkmalsraum – es entsteht ein Mosaik aus Rechtecken mit je einer Vorhersage.

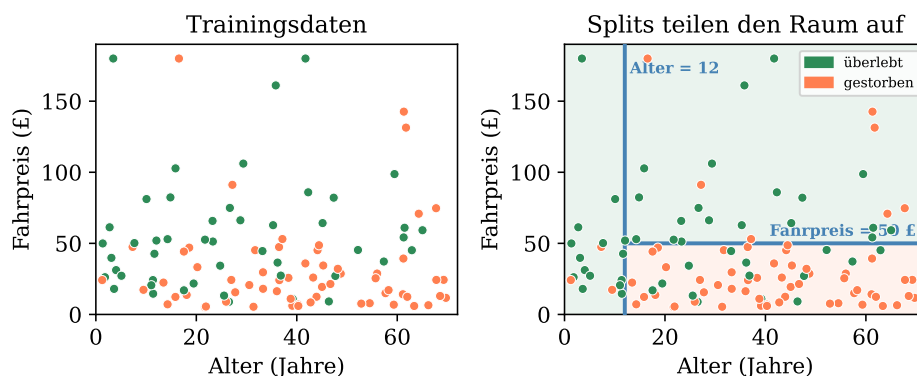


Abbildung 2: **Links:** Trainingsdaten (*überlebt* / *gestorben*). **Rechts:** Der Baum teilt den Merkmalsraum durch achsenparallele Schnitte auf.

Overfitting vermeiden

Ein Baum, der beliebig tief wächst, lernt die Trainingsdaten *auswendig* – er *overfittet*. Man begrenzt deshalb die **maximale Tiefe** (`max_depth`) oder die **minimale Blattgröße** (`min_samples_leaf`). Noch robuster wird es mit einem **Random Forest**: viele Bäume stimmen gemeinsam ab.

Zusammengefasst: Ein Entscheidungsbaum stellt nacheinander Ja/Nein-Fragen und teilt so die Daten in immer reinere Gruppen. Der Algorithmus wählt automatisch die informativsten Fragen. Entscheidungsbäume sind leicht interpretierbar, neigen aber zu Overfitting – deshalb begrenzt man ihre Tiefe oder kombiniert viele Bäume zu einem *Random Forest*.